

$$\min_{\mathbf{S}} - \frac{|\mathbf{T}|}{\sum_{j=1}^{} \ell(\text{LLM}([\mathbf{G}; \mathbf{S}])_j; \mathbf{T}_j)}$$

Source: (GCG; Zou et al., 2023). See also: (GBDA; Gu et al., 2021), (AutPrompt; Shin et al., 2020), (PEZ; Ven et al., 2023).

Jailbreaking *attack strategies.*

Token-based attacks: Greedy coordinate gradient (GCG)

Denote:

$$[\mathbf{G}; \mathbf{S}] := x_{1:n}$$



$$\text{Loss} \coloneqq \mathcal{L}(x_{1:n})$$

Algorithm 1 Greedy Coordinate Gradient

Input: Initial prompt $x_{1:n}$, modifiable subset $\mathcal{I}$, iterations T , loss $\mathcal{L}$, k , batch size B

repeat T times

for $i \in \mathcal{I}$ **do**

$\mathcal{X}_i := \text{Top-}k(-\nabla_{e_{x_i}} \mathcal{L}(x_{1:n}))$

$\triangleright$ Compute top- k promising token substitutions

for $b = 1, \dots, B$ **do**

$\tilde{x}_{1:n}^{(b)} := x_{1:n}$

$\triangleright$ Initialize element of batch

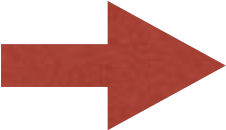
$\tilde{x}_i^{(b)} := \text{Uniform}(\mathcal{X}_i)$, where $i = \text{Uniform}(\mathcal{I})$

$\triangleright$ Select random replacement token

$x_{1:n} := \tilde{x}_{1:n}^{(b^*)}$, where $b^* = \text{argmin}_b \mathcal{L}(\tilde{x}_{1:n}^{(b)})$

$\triangleright$ Compute best replacement

Output: Optimized prompt $x_{1:n}$

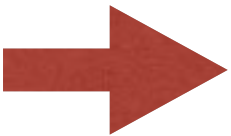


S

 $\dot{z}$

$\mathcal{X}_i =$ - go over all the tokens

- substitute the token, compute gradient at position i
- choose top k tokens



- Choose one token in $\mathcal{X}_i$ via
Greedy search

gradient computation (backward pass)



greedy search
(only needs loss values - forward pass)

Jailbreaking *attack strategies*.

Token-based attacks: **Greedy coordinate gradient** (GCG)

$$\min_{\mathbf{S}} - \sum_{j=1}^{|\mathbf{T}|} \underbrace{\ell(\text{LLM}([\mathbf{G}; \mathbf{S}])_j; \mathbf{T}_j)}_{\text{Loss} := \mathcal{L}(x_{1:n})} \quad \text{Denote: } [\mathbf{G}; \mathbf{S}] := x_{1:n}$$

Algorithm 1 Greedy Coordinate Gradient

Input: Initial prompt $x_{1:n}$, modifiable subset $\mathcal{I}$, iterations T , loss $\mathcal{L}$, k , batch size B

repeat T times

 for $i \in \mathcal{I}$ do

$\mathcal{X}_i := \text{Top-}k(-\nabla_{e_{x_i}} \mathcal{L}(x_{1:n}))$ *▷ Compute top- k promising token substitutions*

 for $b = 1, \dots, B$ do

$\tilde{x}_{1:n}^{(b)} := x_{1:n}$ *▷ Initialize element of batch*

$\tilde{x}_i^{(b)} := \text{Uniform}(\mathcal{X}_i)$, where $i = \text{Uniform}(\mathcal{I})$ *▷ Select random replacement token*

$x_{1:n} := \tilde{x}_{1:n}^{(b^*)}$, where $b^* = \text{argmin}_b \mathcal{L}(\tilde{x}_{1:n}^{(b)})$ *▷ Compute best replacement*

Output: Optimized prompt $x_{1:n}$

gradient computation (backward pass)

greedy search
(only needs loss values - forward pass)

Jailbreaking *attack strategies*.

Token-based attacks: **Adaptive random search**

$$\min_{\mathbf{S}} - \sum_{j=1}^{|\mathbf{T}|} \ell(\text{LLM}([\mathbf{G}; \mathbf{S}])_j; \mathbf{T}_j)$$